字串匹配

找出樣式在文字中的所有出現位置，包含重疊。比較直接搜尋、滾動雜湊、有限自動機與 KMP。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

先定義輸出與邊界情況，再追蹤篩選、狀態轉移與前綴回退；區分正確性與執行時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

字串比對問題

文字 T 長 n，樣式 P 長 m。以 0 起算，輸出每個使 T[s:s+m]=P 的位置 s，包含重疊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

邊界契約

空樣式匹配 n+1 個邊界 0..n。非空樣式若 m>n 則沒有匹配。以下先假設 1≤m≤n。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

四種方法

樸素法直接比較。Rabin–Karp 以雜湊篩選。DFA 預先建立轉移表。KMP 用前綴函數按需回退。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

樸素比對

對每個 s=0..n−m，依序比較 P[j] 與 T[s+j]。全部相等則輸出 s。每次只將 s 加一，保留重疊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

樸素法逐字示例

T=ababa，P=aba。s=0 比較 a、b、a，成功；s=1 首字 b≠a；s=2 比較 a、b、a，成功。輸出 0、2。

Naive / 樸素

```
T   a b a b a
P   a b a     → 0
P     a b a   → fail
P       a b a → 2
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

樸素法成本

最多 $(n-m+1)m$ 次字元比較，最壞時間 $O((n-m+1)m)$，額外空間 $O(1)$。重複字元可逼近最壞情況。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rabin–Karp 指紋

令字元值在 0..d−1，長度 m 的字串雜湊為 Σ x[j]d^(m−1−j) mod q。相等字串必有相等雜湊，反向不保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

滾動雜湊

令 h=d^(m−1) mod q。移出字元 a、加入 b，更新 t′=(d(t−a h)+b) mod q。將負餘數正規化到 0..q−1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

滾動公式從哪裡來？

視窗 W=T[s:s+m]、新字元 c=T[s+m]。把字元視為數值，令未取模的多項式 H(W)=Σ(j=0..m−1) W[j]d^(m−1−j)。

扣除最高位 W[0]d^(m−1)，乘 d，再加 c：

H(T[s+1:s+m+1]) = d(H(W)−W[0]d^(m−1))+c。

逐項移位得到下一視窗。對 q 取模，代入 t_s=H(W) mod q、h=d^(m−1) mod q，即得 t_(s+1)=(d(t_s−W[0]h)+c) mod q；q≥2 不必是質數。

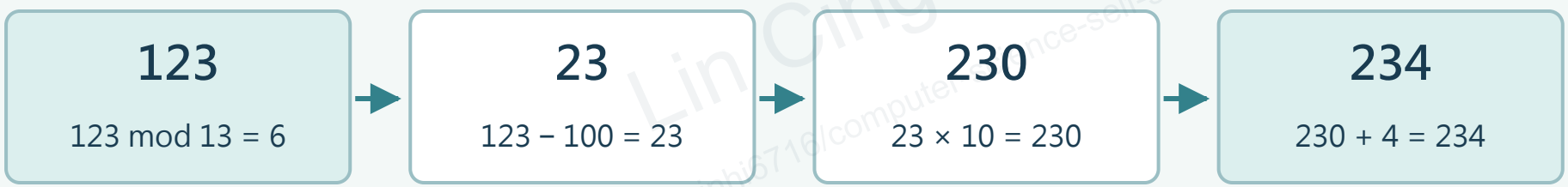
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

滾動掃描的迴圈不變量

處理視窗 s 之前，t_s 等於當前視窗 T[s:s+m] 的 hash；p 是 P 的 hash。初始化時用 Horner 法算出 p 和 t_0。若 s<n−m，前頁恆等式保證更新後的不變量對 s+1 仍成立；最後一格不讀 T[n]。

相等字串必有相同 hash，故真匹配不會被篩掉；hash 相等時再逐字確認，故碰撞不會造成誤報。例中中間差可為負，最後正規化至 0..q−1。

d=10、q=13、m=3：視窗 123 → 234



h=100 mod 13=9；(10·(6−1·9)+4) mod 13=0。

負數餘式須正規化；234 mod 13=0。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

滾動雜湊示例

d=10、q=13、m=2，T=1234、P=34。視窗 12、23、34 的雜湊依序為 12、10、8。樣式雜湊為 8，最後逐字確認。

視窗向右移動一格



移除 1 → 乘 10 → 加入 3

$(10\cdot(12 - 1\cdot10) + 3) \bmod 13 = 10$

移除 2 → 乘 10 → 加入 4

$(10\cdot(10 - 2\cdot10) + 4) \bmod 13 = 8$

h = 10；先前雜湊可取模，結果不變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

雜湊碰撞與成本

當 $1 \le m \le n$，且模運算為常數時間，令 H 為所有雜湊相等視窗數，包含真匹配。逐字確認的時間為 $O(n+m+mH)$，最壞為 $\Theta(n+m+m(n-m+1))$。

$T=a^n$、$P=a^{\lfloor n/2 \rfloor}$ 時，每個視窗都真匹配；即使沒有假碰撞，確認仍需 $\Theta(nm)$。

若隨機雜湊使每個不相等視窗的碰撞機率至多 $1/q$，真匹配數為 V，期望時間為 $O(n+m+mV+mn/q)$。接近線性還須限制真匹配的確認成本，例如 $V=O(1)$ 且 $q \ge m$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

隨機 base 雜湊：隨機化對象與條件

固定輸入後，在質數 q 的有限域 F_q 均勻選 b；字元編碼在 F_q 內互異。長度 m 的字串 X 使用 $H_b(X)=\Sigma(j=0..m-1)\ X[j]\cdot b^{(m-1-j)}$。採用 $b^0=1$，包括 b=0。

對固定相異 X、Y，差 $D(b)=H_b(X)-H_b(Y)$ 是非零多項式，次數至多 m−1。域上非零多項式最多有其次數個根，所以碰撞機率 $\le \min(1,(m-1)/q)$，一般不能直接寫成 1/q。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

多項式根數與期望成本

根數理由：若 r 是根，因式定理可寫 $D(x)=(x-r)Q(x)$；其餘不同根都必為 Q 的根。反覆提出根，根數不能超過次數。

令 $N=n-m+1$、真匹配數 z。對每個非匹配窗口設假命中指示變數，期望線性相加，無須窗口碰撞事件獨立：$E[f]\le(N-z)\min(1,(m-1)/q)$。逐字確認版本的期望時間為 $O(n+m+mz+m(N-z)\min(1,(m-1)/q))$，假設域運算 $O(1)$。真匹配驗證成本 mz 不會消失。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一次假命中與兩次真匹配

取數字編碼、q=11、b=10、m=2，P=12、T=122312。H_b(xy)=10x+y mod 11；P 的 hash 是 1。

五個窗口依序為 12、22、23、31、12，hash 為 1、0、1、9、1。窗口 23 的 hash 與 P 相等卻字串不同，逐字確認後拒絕；位置 0、4 是真匹配。

q=11，b=10：雜湊相同仍需逐字確認

s = 0	s = 1	s = 2	s = 3	s = 4
12 → 1	22 → 0	23 → 1	31 → 9	12 → 1
真匹配	略過	假命中	略過	真匹配

P=12 的 hash=1；核對後輸出 0、4，拒絕 23。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一例子的確切碰撞機率

現在不固定 b=10，而在 F_{11} 均勻選 b。三個非匹配窗口 22、23、31 與 P=12 相碰的方程分別為 b=0、b=10、b=6；每個窗口碰撞機率都是 1/11。

令各窗口碰撞指示變數為 I_1、I_2、I_3；$E[f]=E[I_1]+E[I_2]+E[I_3]=3/11$。事件是否互相獨立不影響這個加總。當 b=10，只有 23 假命中；仍須驗證兩個真匹配。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

自動機狀態

狀態 q 是已讀文字的最長後綴同時為 P 前綴的長度，$0 \le q \le m$。讀入 c 後更新 $\delta(q,c)$。若 $q=m$，輸出目前結尾位置減 m 加一。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DFA 轉移定義

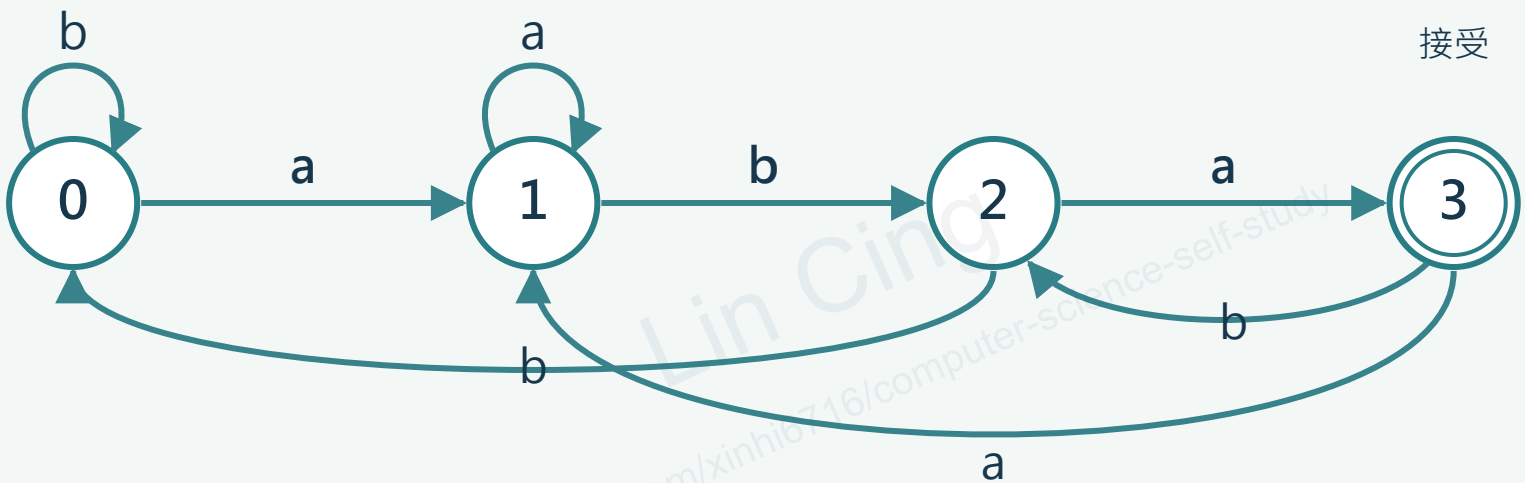
$\delta(q,c)=\max\{k \le m : P[:k] \text{ 是 } P[:q]+c \text{ 的後綴}\}$。包含接受狀態 m 的轉移，才能找出重疊匹配。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DFA 轉移表

P=aba，字母表 {a,b}。各狀態的 (a,b) 轉移為 0:(1,0)、1:(1,2)、2:(3,0)、3:(1,2)。掃描 O(n)，表佔 O(m|Σ|)。

P = aba：箭頭上的字元決定下一個狀態



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從最長後綴得到自動機狀態

令 P⟨k⟩ 表示長度 k 的樣式前綴，P⟨0⟩=ε；P[i] 則表示從 0 起算的第 i 個字元。σ(x) 是同時為 x 後綴的最長樣式前綴長度。狀態 q=σ(x)，轉移定義 δ(q,a)=σ(P⟨q⟩a)。

核心等式是 δ(σ(x),a)=σ(xa)：不必保留整段 x，就能決定加入 a 後的狀態。

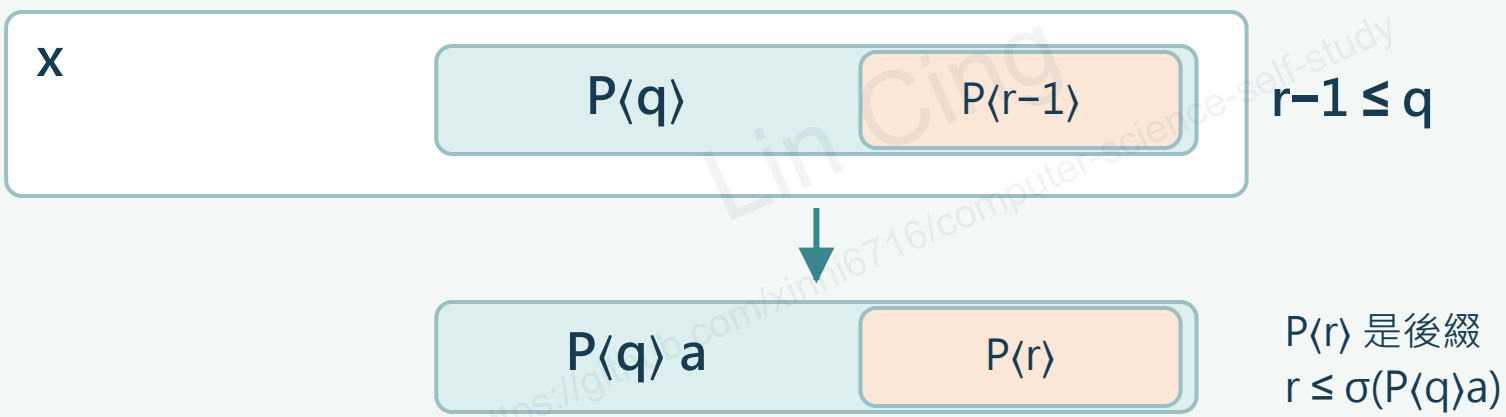
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

狀態等式：第一個方向

設 q=σ(x)、r=σ(xa)。若 r>0，從 P⟨r⟩ 去掉末字後，P⟨r−1⟩ 是 x 的後綴，故 r−1≤q。

P⟨q⟩ 與 P⟨r−1⟩ 都是 x 的後綴，較短者是較長者的後綴；因此 P⟨r−1⟩ 是 P⟨q⟩ 的後綴，再接 a 得 P⟨r⟩ 是 P⟨q⟩a 的後綴。故 r≤σ(P⟨q⟩a)。若 r=0，此不等式顯然成立。

先比較 x 的兩個後綴，再把同一字元 a 接回



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

狀態等式：反向與匹配正確性

P⟨q⟩ 是 x 的後綴，所以 P⟨q⟩a 是 xa 的後綴；P⟨q⟩a 的任何後綴也是 xa 的後綴。故 σ(P⟨q⟩a)≤σ(xa)=r。合併得兩者相等。

初始 σ(ε)=0；每讀一字用等式歸納，狀態恰為 σ(已讀文字)。狀態 m 當且僅當 P 是已讀文字的後綴，這恰好對應一個匹配。接受後仍需轉移，才能找到重疊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

用 π 建立整張 DFA 表

令 π[q−1] 是 P⟨q⟩ 的最長真 border 的長度。對有限字母表 Σ，m≥1：

• q=0：若 a=P[0]，δ(0,a)=1；否則為 0。 • 1≤q≤m：若 q<m 且 a=P[q]，δ(q,a)=q+1；否則 δ(q,a)=δ(π[q−1],a)。

注意 q=m 時不能讀 P[m]，直接使用第二個分支；π[q−1]<q，所以依賴的表列已建立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為什麼建表可以沿 border 回退？

當直接延伸失敗，長度 q+1 的候選不成立；任何較短且非空的匹配 P⟨k⟩，移去末字後，P⟨k−1⟩ 必是 P⟨q⟩ 的真 border。所有真 border 都沿 π 鏈出現，因此以最長真 border 開始重新處理同一字元 a 不會漏掉候選；k=0 永遠可行。

計算 π 用 O(m) 時間及空間，m+1 列、每列 |Σ| 個常數時間格子合計 O(m|Σ|) 時間與空間；假設可 O(1) 存取字母與表格。掃描另需 O(n) 與輸出成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

前綴函數

π[i] 是 P[:i+1] 的最長真前綴且同時是後綴的長度，$0 \leq \pi[i] \leq i$。真前綴排除整個字串。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

前綴函數示例

P=ababaca，索引 0..6；π=[0,0,1,2,3,0,1]。字元 a b a b a c a 對應 0 0 1 2 3 0 1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逐字建立前綴函數

i=0:a→0；1:b 與 a 不同→0；2:a=a→1；3:b=b→2；4:a=a→3；5:c≠b，3→1，c≠b，1→0，c≠a→0；6:a=a→1。

i = 5：比較 c 時，候選邊界反覆回退

索引	0	1	2	3	4	5	6	
	a	b	a	b	a	c	a	
k = 3			a	b	a	b ≠ c		3 → 1
k = 1					a	b ≠ c		1 → 0
k = 0						a ≠ c		π[5] = 0

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

回退為何有效

若已有長度 k 匹配但下一字不符，較短候選必須是 P[:k] 的邊界。最長候選為 π[k−1]。反覆回退直到匹配或 k=0。例：P=ababaca、k=5、下一字元 x，回退鏈是 5→3→1→0，不能只回退一次。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

border 鏈為什麼列盡所有候選？

設 B 是字串 X 的最長真 border。任一較短真 border C 同時是 X 的前綴與後綴，也是 B 的前綴與後綴，因此是 B 的 border；反過來 B 的 border 也是 X 的 border。

從長度 |X| 開始反覆取 π[length−1]，每次嚴格變小，必以 0 結束；由上述雙向包含，此鏈恰列出所有真 border 的長度，沒有漏項。

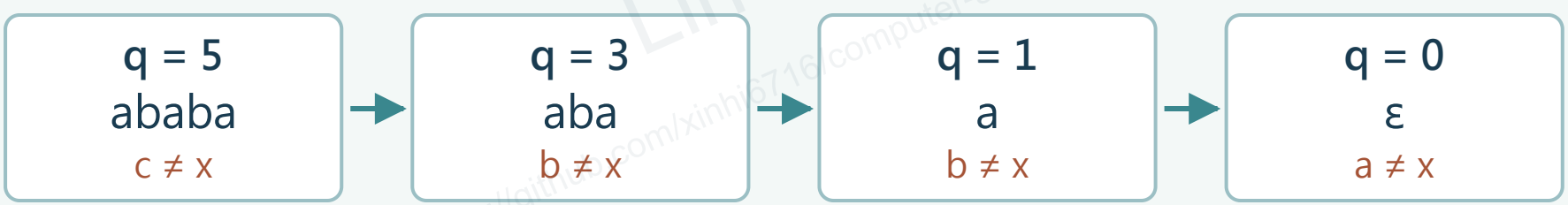
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

前綴函數：不變量與正確性

若 m=0，π 為空表，不讀取樣式字元。若 m≥1，先設 π[0]=0：單一字元沒有非空真 border，這是歸納基例。接著對 i=1,⋯,m−1，已知 π[0..i−1]，令 k=π[i−1]。當 P[k]≠P[i] 且 k>0，沿 k←π[k−1] 反覆回退；同一 P[i] 不動。

每次回退留下的候選依長度遞減且列盡，因此第一個能延伸者給最長 border，令 π[i]=k+1。若到 k=0 仍不相等，答案是 0；若 P[0]=P[i]，答案是 1。這完成對 i 的歸納。

P = ababaca：對同一字元 x 連續回退



5 → 3 → 1 → 0；在 q=0 比較 P[0]，不符就維持 0。

較短 border 必為最長 border 的 border。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為什麼建 π 只需線性時間？

以 k 為非負勢能。每個 i 最多增加 k 一次、幅度至多 1；每次 while 回退使 k 至少減 1。從 k=0 出發，累計回退次數不超過累計增加量 $\leq m-1$。

每個 i 還有至多一次終止比較與常數工作，因此建表 O(m)，KMP 掃描同理 O(n)。這是跨整趟執行的界，單一字元仍可能觸發多次回退。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

KMP 預處理

π[0]=0。i 從 1 到 m−1，令 k=π[i−1]；只要 k>0 且字元不符，就反覆令 k=π[k−1]；停止回退後若相符則 k 加一；令 π[i]=k。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

KMP 掃描

令 q=0。逐字 c：只要 q>0 且 P[q]≠c，就反覆令 q=π[q−1]；停止回退後若 P[q]=c，q 加一。q=m 時輸出 i−m+1，再設 q=π[m−1]，保留重疊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

KMP 掃描的兩種狀態

假設 $m \geq 1$，令 $\sigma(x)$ 是 x 的後綴中最長樣式前綴的長度（可為 m）；令 $\rho(x)$ 是最長長度小於 m 的樣式前綴後綴長度。

每輪開始的 $q=\rho(x)$，其中 x 是已讀文字；初始 $q=\rho(\varepsilon)=0$。若 $\sigma(x)<m$，則 $\rho(x)=\sigma(x)$；若 $\sigma(x)=m$，則 $\rho(x)=\pi[m-1]$。因此接受後儲存的是最長真 border，不是 m。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一次掃描等價於 DFA 轉移

對同一新字元 a，while 沿 border 鏈從 q 遞減嘗試；第一個可延伸的候選最長，全部失敗則為 0。因此暫存狀態 $r=\delta(q,a)$。

若 $\sigma(x)<m$，$q=\sigma(x)$；若 $\sigma(x)=m$，$q=\pi[m-1]$ 且 $\delta(m,a)=\delta(\pi[m-1],a)$。由狀態等式 $\delta(\sigma(x),a)=\sigma(xa)$，兩種情況均有 $r=\sigma(xa)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為什麼輸出後回退不漏重疊？

r=m 當且僅當樣式為已讀文字 xa 的後綴，因而輸出位置 i−m+1 恰好正確；接著 q←π[m−1]=ρ(xa)。若 r<m，令 q←r=ρ(xa)。兩種情況均恢復下輪不變量。

對 P=aba、T=ababa，位置 2 與 4 分別輸出 0 與 2；兩次接受後都保留 q=1。後續字元從該 border 繼續，因此不漏重疊。

P=aba、T=ababa：先接受，再保留 border

i	0	1	2	3	4
T[i]	a	b	a	b	a
輸入前 q	0	1	2	1	2
r = σ	1	2	3	2	3
輸出後 q	1	2	1	2	1
輸出	–	–	0	–	2

r=3 時輸出，然後 q=π[2]=1。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

KMP 逐字追蹤

T=ababa，P=aba，π=0,0,1。各 i、字元、q：0 a→1；1 b→2；2 a→3，輸出 0 並回退 1；3 b→2；4 a→3，輸出 2 並回退 1。

P = aba：完整匹配後保留長度 1 的邊界

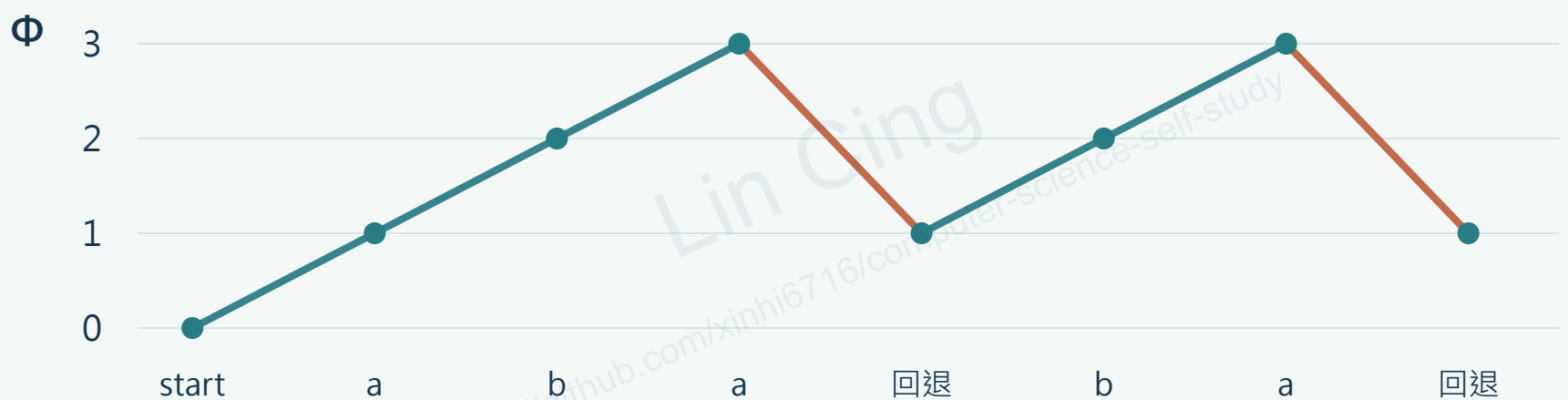
T 的索引	0	1	2	3	4
讀入字元	a	b	a	b	a
狀態 q	1	2	3	2	3
匹配位置			0 3 → 1		2 3 → 1

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

線性時間的攤銷證明

預處理及掃描各自用 q 或 k 為勢能。每個輸入位置至多使勢能增加 1；每次非零回退至少減少 1，故總回退不超過總增加。其他比較每位置 O(1)，總時間 O(m+n)。

勢能 Φ = q：上升至多一次，回退必下降



每次下降至少 1；總回退次數不超過總上升次數。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

KMP 時間與空間

預處理 O(m)，掃描 O(n)，π 表 O(m)。儲存 z 筆輸出另需 O(z)；若串流輸出則不計入工作空間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

四法成本比較

在 $1 \le m \le n$ 下，樸素法最壞 $O(nm)$ 時間及 $O(1)$ 空間；Rabin–Karp 最壞 $O(nm)$ 及 $O(1)$ 工作空間；DFA 掃描 $O(n)$、表 $O(m|\Sigma|)$；KMP $O(n+m)$ 及 $O(m)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

實作陷阱

明確指定字元與索引單位：位元組、Unicode code point 與 grapheme cluster 不同。先處理空樣式；雜湊命中須確認；完整匹配後不可直接重設為零。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

驗算策略

用直接枚舉當 oracle，窮舉小字母表所有短 T、P，比較 Rabin–Karp、DFA、KMP 的完整輸出。測試碰撞、空樣式、m>n 和重疊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

核心概念

已匹配前綴的邊界組成可行回退鏈。KMP 沿此鏈回退，DFA 預算狀態更新，雜湊法以確認步驟保證正確。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

CLRS, Introduction to Algorithms, 3rd ed., chapter 32, pp. 990–1006. MIT 6.046J Fall 2004, Lecture 22.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study